

Base de Données NoSQL

MongoDB

Modéliser les Documents

Objectifs du chapitre :

1. Structurer un document JSON
2. Utiliser les espaces de noms, collections et documents (BSON)
3. Identifier les différences entre la modélisation MongoDB et relationnelle
4. Modéliser les liens entre documents

Table des matières

1	Structure d'un Document JSON	3
1.1	Définition de JSON	3
1.1.1	Caractéristiques principales	3
1.2	Syntaxe de Base	3
1.2.1	Structure d'un objet JSON	3
1.2.2	Types de valeurs JSON	3
1.3	Règles Syntaxiques Complètes	4
1.4	Exemple Complet de Document JSON	5
1.5	Usages de JSON	5
2	Espaces de Noms, Collections et Types de Données	6
2.1	Hiérarchie des Données dans MongoDB	6
2.2	Espace de Nom (Namespace)	6
2.3	Collections	6
2.3.1	Caractéristiques des collections	6
2.4	Types de Données BSON	7
2.4.1	Types de données principaux	7
2.4.2	String	7
2.4.3	Integer (Int32 et Int64)	7
2.4.4	Double	8
2.4.5	Boolean	8
2.4.6	Date	8
2.4.7	Null	9
2.4.8	Array	9
2.4.9	ObjectId	9
2.4.10	Binary Data	10
2.5	Validation de Schéma	11
3	Modélisation MongoDB vs Base de Données Relationnelle	12
3.1	Normalisation vs Dénormalisation	12
3.1.1	Approche relationnelle : La normalisation	12
3.1.2	Approche NoSQL : La dénormalisation	12
3.2	Caractéristiques de MongoDB	12
3.2.1	Différences avec le modèle relationnel	12
3.2.2	Approche orientée requêtes	13
4	Modélisation des Liens	14
4.1	L'Enchâssement (Embedding)	14
4.1.1	Avantages	14
4.1.2	Quand utiliser l'embedding ?	14
4.1.3	Exemple d'embedding	14
4.1.4	Limites de l'embedding	15
4.2	La Liaison (Linking/Referencing)	15
4.2.1	Fonctionnement	15
4.2.2	Quand utiliser le linking ?	15
4.2.3	Exemple de linking	16
4.2.4	Jointure avec \$lookup	16

4.3	Comparaison Embedding vs Linking	17
4.4	Patterns de Modélisation Avancés	17
4.4.1	Pattern Hybride	17
5	Exercices Pratiques	18
5.1	Exercice 1 : Modélisation de Relations	18
5.2	Exercice 2 : Identification des Types	18
5.3	Exercice 4 : Optimisation de Schéma	18

1 Structure d'un Document JSON

1.1 Définition de JSON

Définition : JSON

JSON (JavaScript Object Notation) est un format standard de représentation logique de données, hérité de la syntaxe de création d'objets en JavaScript.

1.1.1 Caractéristiques principales

- **Format texte léger** : peu de caractères de structuration
- **Lisible par les humains** : syntaxe claire et intuitive
- **Extension de fichier** : `.json`
- **Indépendant du langage** : peut être interprété par tout langage via un parser
- **Usage principal** : structurer et transmettre des données sur le web (client ↔ serveur)

1.2 Syntaxe de Base

1.2.1 Structure d'un objet JSON

Un objet JSON repose sur deux éléments essentiels :

Élément	Description
Clé	Chaîne de caractères entourée de guillemets doubles
Valeur	Type de données JSON valide (string, number, boolean, null, object, array)

Règles syntaxiques fondamentales

- Un objet JSON commence par `{` et se termine par `}`
- Les paires clé/valeur sont séparées par une **virgule**
- La clé est suivie de **deux-points** (`:`) pour la distinguer de la valeur
- Les clés doivent être des **chaînes de caractères** entre guillemets

1.2.2 Types de valeurs JSON

JSON supporte trois catégories de valeurs :

1. Types primitifs

- **String** : chaîne de caractères entre guillemets `"texte"`
- **Number** : nombre entier ou décimal `42`, `3.14`
- **Boolean** : valeur logique `true` ou `false`
- **Null** : valeur nulle `null`

2. Objet Liste de paires "clé" : valeur entre accolades, séparées par des virgules.

Exemple : Objet JSON

```
1 {  
2   "stagiaire": {  
3     "prenom": "Amina",  
4     "filierre": "Dev Digital",  
5     "niveau": "1A"  
6   }  
7 }
```

3. Tableau (Array) Ensemble ordonné de valeurs entre crochets [], séparées par des virgules.

Exemple : Tableau JSON

```
1 {  
2   "stagiaires": [  
3     {"prenom": "Amina", "filierre": "Dev Digital", "niveau  
4       ": "1A"},  
5     {"prenom": "Kamal", "filierre": "Infra Digitale", "  
6       niveau": "1A"},  
7     {"prenom": "Sanaa", "filierre": "Infra Digitale", "  
        niveau": "1A"}  
   ]  
}
```

1.3 Règles Syntaxiques Complètes

Validation d'un fichier JSON

Tout fichier JSON bien formé doit respecter les règles suivantes :

1. Soit un **objet** commençant par { et terminant par }
2. Soit un **tableau** commençant par [et terminant par]
3. Les objets/tableaux vides sont valides : {} et []
4. Un objet JSON peut contenir d'autres objets JSON (imbrication)
5. Le format JSON ne tolère pas les éléments croisés
6. Il doit exister un seul **élément racine** par document

1.4 Exemple Complet de Document JSON

Document JSON avec différents types

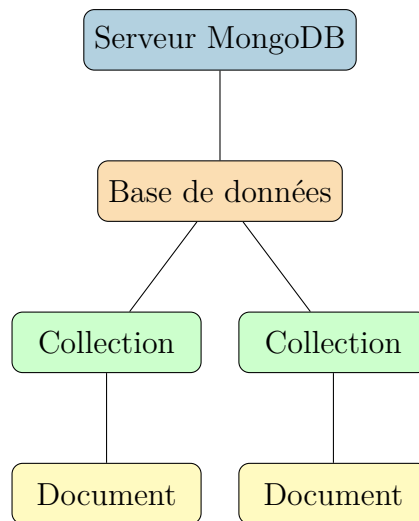
```
1 {  
2   "_id": 12,  
3   "prenom": "Kamal",  
4   "type": "Stagiaire",  
5   "filierre": "Dev Digital",  
6   "groupe": "DD203",  
7   "niveau": "2A",  
8   "option": "Mobile",  
9   "age": 21,  
10  "moyenne": 14.75,  
11  "actif": true,  
12  "telephone": null,  
13  "competences": ["Python", "JavaScript", "MongoDB"],  
14  "adresse": {  
15    "ville": "Casablanca",  
16    "codePostal": "20000"  
17  }  
18 }
```

1.5 Usages de JSON

Domaine	Description
Chargements Asynchrones	Avec AJAX, JSON s'est montré plus adapté que XML pour les échanges asynchrones
APIs REST	Format dominant pour les APIs (Twitter, Facebook, LinkedIn). JSON a supplanté XML dans ce domaine
Bases de données NoSQL	Format natif pour MongoDB, CouchDB, Riak. Également possible de récupérer des réponses JSON depuis les SGBDR
Fichiers de configuration	Utilisé par npm (package.json), VS Code, et de nombreux outils

2 Espaces de Noms, Collections et Types de Données

2.1 Hiérarchie des Données dans MongoDB



2.2 Espace de Nom (Namespace)

Définition : Namespace

Un **espace de nom** (namespace) est la concaténation du nom de la base de données et du nom de la collection, séparés par un point.

Exemples de namespaces

- `myDB.Stagiaires` → Collection “Stagiaires” dans la base “myDB”
- `ecole.cours` → Collection “cours” dans la base “ecole”
- `shop.products.reviews` → Sous-collection (convention de nommage)

2.3 Collections

Définition : Collection

Une **collection** est un conteneur pour les documents. Elle est l'équivalent d'une table dans une base de données relationnelle, mais sans schéma fixe.

2.3.1 Caractéristiques des collections

- **Schéma flexible** : les documents peuvent avoir des structures différentes
- **Création implicite** : créée automatiquement lors du premier insert
- **Pas de jointures natives** : données dénormalisées ou références
- **Index** : possibilité de créer des index pour optimiser les requêtes

2.4 Types de Données BSON

Définition : BSON

BSON (Binary JSON) est le format binaire utilisé par MongoDB pour stocker les documents. Il étend JSON avec des types de données supplémentaires.

2.4.1 Types de données principaux

Type	Numéro	Description
Double	1	Nombre à virgule flottante 64 bits
String	2	Chaîne de caractères UTF-8
Object	3	Document imbriqué
Array	4	Tableau de valeurs
Binary Data	5	Données binaires
ObjectId	7	Identifiant unique 12 octets
Boolean	8	Valeur true ou false
Date	9	Date UTC (millisecondes depuis epoch)
Null	10	Valeur nulle
Int32	16	Entier signé 32 bits
Int64	18	Entier signé 64 bits
Decimal128	19	Décimal haute précision 128 bits

2.4.2 String

Type String

```
1 {  
2   "_id": "1234",  
3   "prenom": "Amina",  
4   "email": "amina@example.com"  
5 }
```

Les strings BSON doivent être encodées en **UTF-8**.

2.4.3 Integer (Int32 et Int64)

Types Integer

```
1 {  
2   "_id": "1234",  
3   "prenom": "Amina",  
4   "age": 19,  
5   "score": NumberLong(999999999)  
6 }
```

- **Int32** : entier signé 32 bits (-2,147,483,648 à 2,147,483,647)
- **Int64** : entier signé 64 bits (NumberLong)

2.4.4 Double

Type Double

```
1 {  
2   "_id": "1234",  
3   "prenom": "Amina",  
4   "moyBaccalaureat": 14.25,  
5   "coefficient": 2.5  
6 }
```

Utilisé pour stocker les valeurs à virgule flottante.

2.4.5 Boolean

Type Boolean

```
1 {  
2   "_id": "1234",  
3   "prenom": "Amina",  
4   "admis": true,  
5   "boursier": false  
6 }
```

2.4.6 Date

Type Date

```
1 {  
2   "prenom": "Ahmed",  
3   "niveau": "1A",  
4   "filiere": "Dev digital",  
5   "DateInscription_1": Date(),  
6   "DateInscription_2": new Date(),  
7   "DateInscription_3": ISODate("2024-01-15T10:30:00Z")  
8 }
```

Résultat stocké :

```
1 {  
2   "_id": ObjectId("62dd1dff1a19f3d7ecc66252"),  
3   "prenom": "Ahmed",  
4   "DateInscription_1": "Mon Jan 15 2024 11:25:03 GMT+0100",  
5   "DateInscription_2": ISODate("2024-01-15T10:25:03.613Z")  
6 }
```

Remarques sur les dates

- `Date()` retourne une **string**
- `new Date()` et `ISODate()` retournent un **objet Date**
- Les dates sont stockées en **millisecondes** depuis le 1er janvier 1970 (epoch)
- Les valeurs négatives représentent les dates **antérieures à 1970**

2.4.7 Null

Type Null

```

1 {
2   "_id": "1234",
3   "prenom": "Amina",
4   "telephone": null,
5   "adresse_seconde": null
6 }
```

Utilisé pour représenter l'absence de valeur ou une valeur inconnue.

2.4.8 Array

Type Array

```

1 {
2   "_id": "1234",
3   "prenom": "Kamal",
4   "niveau": "2A",
5   "option": "Mobile",
6   "skills": ["python", "javascript", "php", "mongodb"],
7   "notes": [15.5, 14.0, 16.25, 13.75],
8   "projets": [
9     {"nom": "App Mobile", "note": 17},
10    {"nom": "Site Web", "note": 15}
11  ]
12 }
```

Un tableau peut contenir des valeurs de **types différents**, y compris des objets.

2.4.9 ObjectId

Définition : ObjectId

L'**ObjectId** est un identifiant unique de 12 octets généré automatiquement par MongoDB pour chaque document si le champ `_id` n'est pas spécifié explicitement.

Structure d'un ObjectId (12 octets) :

4 octets	5 octets	3 octets
Timestamp Unix	Valeur aléatoire	Compteur incrémental

ObjectId - Génération automatique vs explicite**Sans `_id` spécifié :**

```
1 db.stagiaires.insertOne({
2   "prenom": "Ahmed",
3   "niveau": "1A",
4   "filier": "Dev digital"
5 })
```

Résultat :

```
1 {
2   "_id": ObjectId("62dd128a1a19f3d7ecc6624f"),
3   "prenom": "Ahmed",
4   "niveau": "1A",
5   "filier": "Dev digital"
6 }
```

Avec `_id` explicite :

```
1 db.stagiaires.insertOne({
2   "_id": 1234,
3   "prenom": "Alaa",
4   "niveau": "2A",
5   "filier": "Dev digital"
6 })
```

Résultat :

```
1 {
2   "_id": 1234,
3   "prenom": "Alaa",
4   "niveau": "2A",
5   "filier": "Dev digital"
6 }
```

2.4.10 Binary Data**Type Binary Data**

```
1 {
2   "_id": "1234",
3   "prenom": "Amina",
4   "photo": BinData(0, "iVBORw0KGgoAAAANSU..."),
5   "signature": BinData(0, "base64encodeddata...")
6 }
```

Utilisé pour stocker des fichiers, images, ou toute donnée binaire.

2.5 Validation de Schéma

Bien que MongoDB soit schema-less, il est possible de définir des règles de validation :

Validation de schéma avec JSON Schema

```
1 db.createCollection("stagiaires", {
2   validator: {
3     $jsonSchema: {
4       bsonType: "object",
5       required: ["prenom", "nom", "filier"],
6       properties: {
7         prenom: {
8           bsonType: "string",
9           description: "Prenom requis - string"
10        },
11        age: {
12          bsonType: "int",
13          minimum: 16,
14          maximum: 65,
15          description: "Age entre 16 et 65"
16        },
17        email: {
18          bsonType: "string",
19          pattern: "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-
20            ]+\\. [a-zA-Z]{2,}$"
21        }
22      }
23    }
24  })
```

3 Modélisation MongoDB vs Base de Données Relationnelle

3.1 Normalisation vs Dénormalisation

3.1.1 Approche relationnelle : La normalisation

Dans les bases de données relationnelles :

- La modélisation repose sur la **normalisation** des structures de données
- Objectif : éviter toute duplication de l'information
- Les requêtes se basent essentiellement sur les **jointures**
- Le modèle des données impose la manière d'écriture des requêtes

3.1.2 Approche NoSQL : La dénormalisation

Définition : Dénormalisation

La **dénormalisation** consiste à regrouper plusieurs tables liées par des références en une seule structure, en éliminant les jointures. Elle favorise la **redondance des données**.

Objectifs de la dénormalisation

- Améliorer les **performances en lecture** sur les tables considérées
- Réduire le **temps de réponse** des requêtes
- Simplifier la **structure des requêtes** (pas de jointures)
- Optimiser pour les contextes **lecture-intensive**

3.2 Caractéristiques de MongoDB

Les bases de données NoSQL se caractérisent par :

- L'abandon des **jointures** et **transactions** complexes
- La priorité au **temps de réponse court**
- Des **performances optimales** en lecture
- Un **schéma flexible** (schema-less)

3.2.1 Différences avec le modèle relationnel

Contraintes de normalisation non respectées dans MongoDB

- **Attributs non atomiques** : première forme normale (1NF) non respectée
- **Données redondantes** : deuxième forme normale (2NF) non respectée
- **Dépendances transitives** : troisième forme normale (3NF) non obligatoire

3.2.2 Approche orientée requêtes

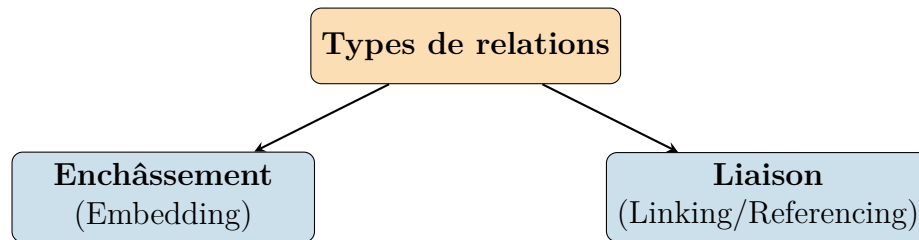
Principe fondamental

Dans MongoDB, c'est le **type de requêtes** qu'on désire élaborer sur les données qui détermine le schéma des données, et non l'inverse.

“On détermine le schéma des données suivant l'utilisation des données par l'application.”

4 Modélisation des Liens

MongoDB propose deux approches pour modéliser les relations entre documents :



4.1 L'Enchâssement (Embedding)

Définition : Embedding

L'**enchâssement** consiste à imbriquer partiellement ou totalement un document B dans un document A. On parle de **document autonome** (self-contained document).

4.1.1 Avantages

- **Évite les jointures** : toute l'information est dans un seul document
- **Performances optimales en lecture** : une seule opération de base de données
- **Atomicité** : mise à jour atomique du document entier
- **Cohérence des données** : données associées stockées ensemble

4.1.2 Quand utiliser l'embedding ?

Cas d'utilisation de l'embedding

- Documents **petits** et qui n'ont pas tendance à grandir
- Relation **un-à-un** (one-to-one)
- Relation **un-à-peu** (one-to-few)
- Données **toujours accédées ensemble**
- Contexte privilégiant la **lecture** à la mise à jour

4.1.3 Exemple d'embedding

Relation Stagiaire - Filière (Embedding)

```
1 {
2   "_id": "1234",
3   "prenom": "Amina",
4   "nom": "El Amrani",
5   "filiere": {
6     "_id": "DD",
7     "intitule": "Developpement Digital",
```

```
8       "duree": "2 ans"
9     },
10    "adresse": {
11      "rue": "123 Avenue Hassan II",
12      "ville": "Casablanca",
13      "codePostal": "20000"
14    }
15  }
```

4.1.4 Limites de l'embedding

Limitations importantes

- Taille maximale d'un document : **16 Mo**
- Duplication des données si l'entité imbriquée est partagée
- Mises à jour complexes si les données imbriquées changent fréquemment
- Profondeur d'imbrication limitée à **100 niveaux**

4.2 La Liaison (Linking/Referencing)

Définition : Linking

La **liaison** consiste à inclure les liens ou références d'un document dans un autre, reprenant le concept de **clé étrangère** des bases relationnelles.

4.2.1 Fonctionnement

La récupération des données se fait en deux étapes :

1. Une première requête pour récupérer le document avec la référence
2. Une deuxième requête pour récupérer les données référencées

4.2.2 Quand utiliser le linking ?

Cas d'utilisation du linking

- Relation **plusieurs-à-plusieurs** (many-to-many)
- Relation **un-à-beaucoup** (one-to-many) avec beaucoup d'éléments
- Documents qui peuvent **dépasser 16 Mo**
- Données partagées entre **plusieurs documents**
- Données qui changent **fréquemment**
- Nécessité d'accéder aux données **indépendamment**

4.2.3 Exemple de linking

Relation Stagiaire - Filière (Linking)

Collection : filieres

```
1 {  
2   "_id": "DD",  
3   "intitule": "Developpement Digital",  
4   "duree": "2 ans",  
5   "responsable": "M. Benali"  
6 }
```

Collection : stagiaires

```
1 {  
2   "_id": "1234",  
3   "prenom": "Amina",  
4   "nom": "El Amrani",  
5   "filiere_id": "DD"  
6 }
```

4.2.4 Jointure avec \$lookup

MongoDB permet de faire des jointures via l'opérateur `$lookup` dans le pipeline d'agrégation :

Exemple de \$lookup

```
1 db.stagiaires.aggregate([  
2   {  
3     $lookup: {  
4       from: "filieres",  
5       localField: "filiere_id",  
6       foreignField: "_id",  
7       as: "filiere_info"  
8     }  
9   }  
10  ])
```

4.3 Comparaison Embedding vs Linking

Critère	Embedding	Linking
Performance lecture	Excellente (une requête)	Moyenne (plusieurs requêtes)
Performance écriture	Variable	Meilleure pour données partagées
Duplication	Oui (données copiées)	Non (références uniquement)
Cohérence	Automatique	À gérer manuellement
Taille document	Limitée à 16 Mo	Pas de limite pratique
Relations	1-1, 1-few	1-many, many-many
Complexité requêtes	Simple	Plus complexe (\$lookup)

4.4 Patterns de Modélisation Avancés

4.4.1 Pattern Hybride

Combiner embedding et linking selon les besoins :

Pattern Hybride : Commande avec produits

```

1 {
2   "_id": ObjectId("..."),
3   "date_commande": ISODate("2024-01-15"),
4   "client": {
5     "_id": "C001",
6     "nom": "Entreprise ABC"
7   },
8   "lignes": [
9     {
10      "produit_id": "P001",
11      "nom_produit": "Ordinateur",
12      "quantite": 5,
13      "prix_unitaire": 8000
14    },
15    {
16      "produit_id": "P002",
17      "nom_produit": "Souris",
18      "quantite": 10,
19      "prix_unitaire": 150
20    }
21  ],
22  "total": 41500
23 }
```

5 Exercices Pratiques

5.1 Exercice 1 : Modélisation de Relations

Soit le schéma relationnel suivant :

- AUTEUR (id_auteur, nom, prenom, nationalite)
- LIVRE (id_livre, titre, annee, #id_auteur)
- EMPRUNT (id_emprunt, date_emprunt, date_retour, #id_livre, #id_membre)
- MEMBRE (id_membre, nom, email)

Questions :

1. Proposer une modélisation MongoDB avec **embedding**
2. Proposer une modélisation MongoDB avec **linking**
3. Quelle approche privilégieriez-vous ? Justifiez.

5.2 Exercice 2 : Identification des Types

Identifier le type BSON de chaque valeur dans le document suivant :

```
1 {
2   "_id": ObjectId("507f1f77bcf86cd799439011"),
3   "nom": "Dupont",
4   "age": 25,
5   "salaire": 3500.50,
6   "actif": true,
7   "departement": null,
8   "competences": ["Java", "Python"],
9   "dateEmbauche": ISODate("2023-06-15T00:00:00Z"),
10  "adresse": {
11    "ville": "Paris",
12    "codePostal": "75001"
13  }
14 }
```

5.3 Exercice 4 : Optimisation de Schéma

Une application de blog stocke actuellement les articles avec tous les commentaires imbriqués. Certains articles ont plus de 10 000 commentaires.

Questions :

1. Quels problèmes cette modélisation peut-elle poser ?
2. Proposez une modélisation optimisée